

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	MRUDHULA K	Reg. No.:	192524335
Department:	B.TECH AIDS	Date:	15.08.26

Code of Conduct:

I MRUDHULA K (192524335) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

MRUDHULA K
Signature of the student:

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyse the sequence of operations and the role of the ALU in processing data.

SOLUTION:

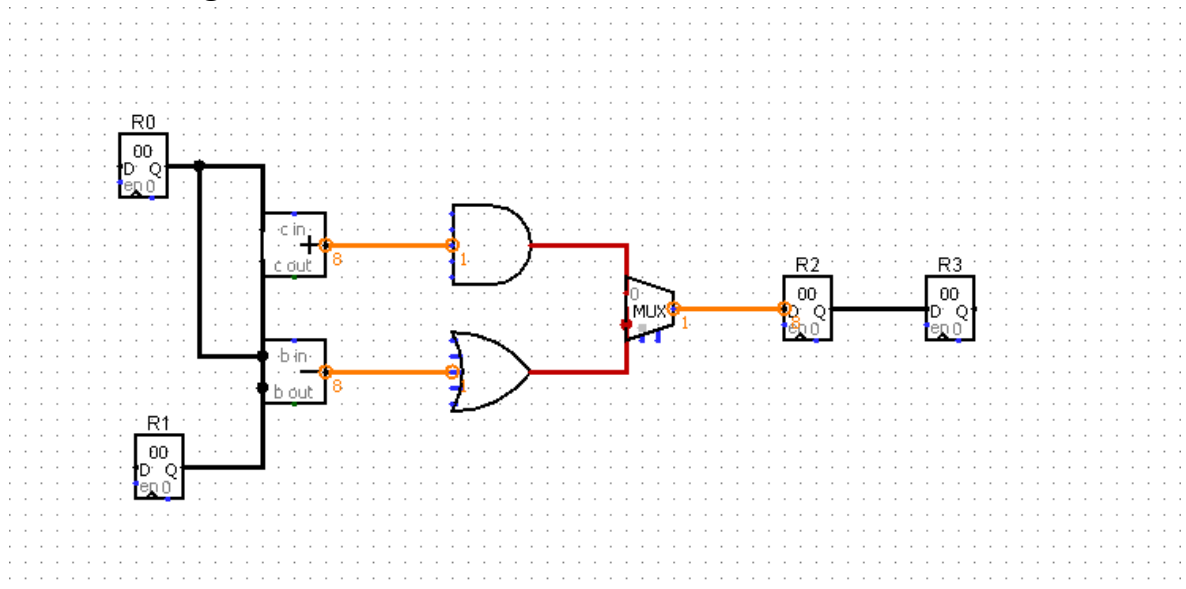
Aim:

To design and simulate register transfer operations with arithmetic and logic operations using multiple registers and an ALU.

Algorithm:

- Initialize registers R1, R2 and R3.
- Load data into R1 and R2.
- Transfer data between registers using control signals.
- Perform arithmetic operations such as addition and subtraction using the ALU.
- Perform logical operations such as AND and OR.
- Store the ALU output in the destination register.
- Display the sequence of register transfers and results.

Experimental Design:



Observation Table :

Operation	R0	R1	Expected R2	Logisim R2
ADD	10	5	15	15
SUB	10	5	5	5
AND	10	5	0	0
OR	10	5	15	15

Result:

Register transfer, arithmetic, and logical operations were successfully simulated, demonstrating the role of the ALU in processing data.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyse how multiple bus structures improve parallel data transfer and reduce bottlenecks.

SOLUTION:**Aim:**

To model and compare single-bus and multi-bus computer organizations for data transfer between registers, memory, and I/O.

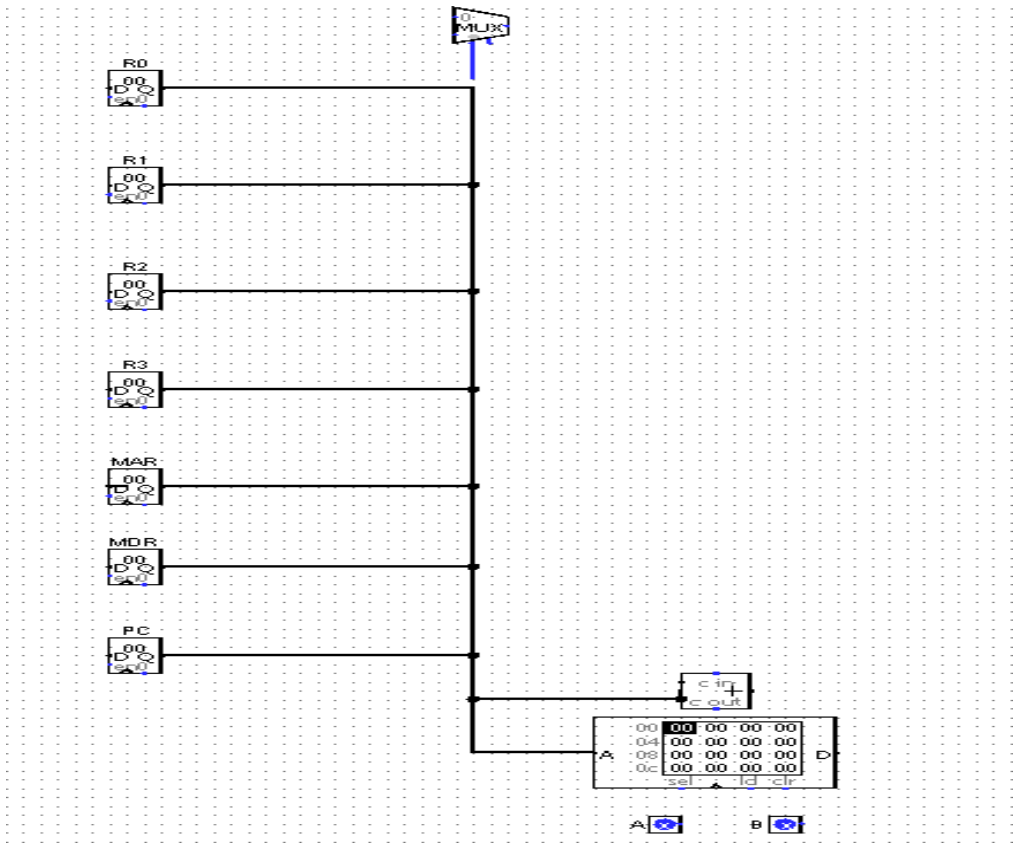
Algorithm:

- Initialize registers, memory, and I/O units.
- Implement data transfer using a single-bus structure.
- Record the number of cycles/time required.
- Implement the same operations using a multi-bus structure.
- Allow simultaneous data transfers wherever possible.
- Compare execution time and efficiency.
- Analyse the reduction in bottlenecks.

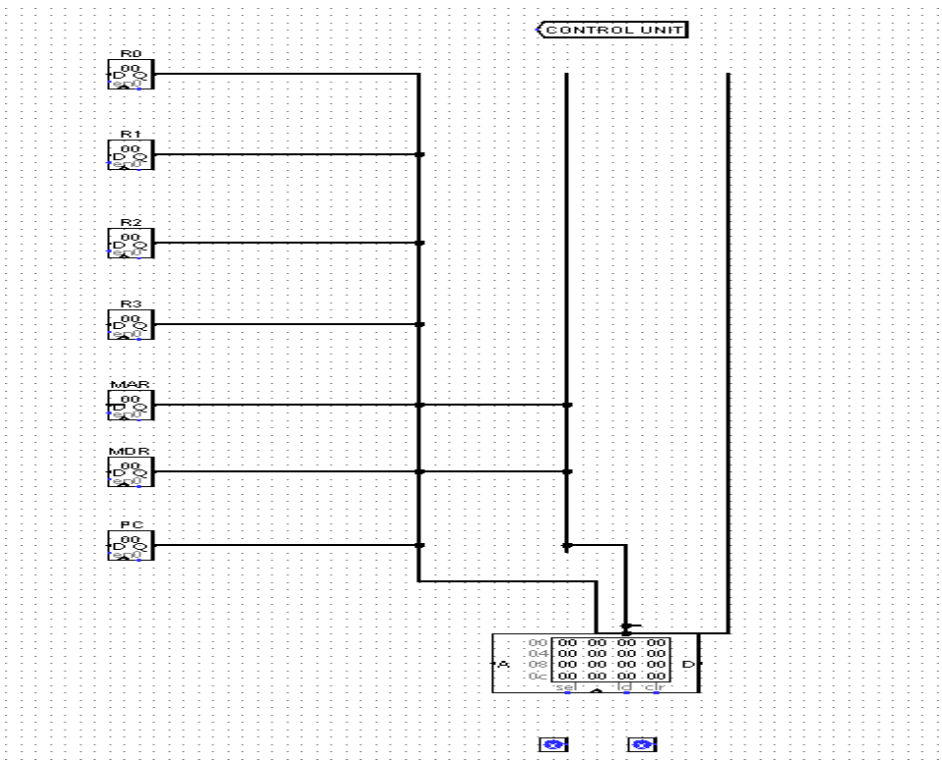
Objectives

- To understand the concept of bus organization in a computer system.
- To design a single-bus architecture using registers, memory, I/O, and a control unit.
- To design a multi-bus architecture using multiple independent buses.
- To represent data transfer between registers, memory, and I/O.
- To compare single-bus and multi-bus organizations.
- To analyze how multiple buses reduce data-transfer bottlenecks and improve parallelism.

Single-Bus Architecture:



Multi-bus architecture:



Analysis:

Operation	Single-Bus	Multi-Bus	Observation
$R1 \rightarrow R2$	1 transfer	1 transfer	Similar
$R1 \rightarrow \text{MAR}$	1 transfer	1 transfer	Similar
Memory $\rightarrow$ MDR	1–2 transfers	1 transfer	Multi-bus can reduce transfer contention
$R1 + R2 \rightarrow \text{destination}$	Multiple sequential transfers	Parallel operand transfers	Multi-bus is more efficient

Result:

Both bus organizations were successfully modelled. The multi-bus system provided faster and more efficient data transfer by allowing parallel transfers and reducing bus bottlenecks.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

SOLUTION:

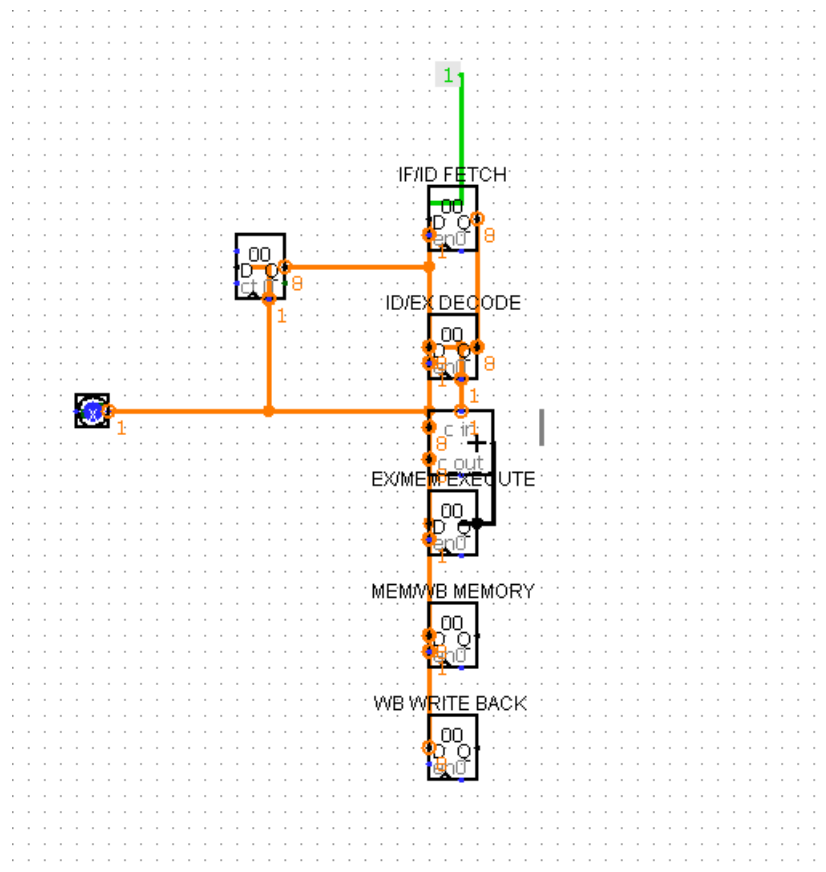
Aim:

To simulate a 5-stage instruction pipeline and compare its performance with a non-pipelined system.

Algorithm:

- Define the instruction sequence.
- Divide instruction execution into five stages:
 - **IF** – Instruction Fetch
 - **ID** – Instruction Decode
 - **EX** – Execute
 - **MEM** – Memory Access
 - **WB** – Write Back
- Execute instructions sequentially in the non-pipelined system.
- Execute instructions simultaneously in different pipeline stages.
- Calculate total execution cycles.
- Calculate speedup and throughput.
- Compare pipelined and non-pipelined performance.

Experimental design:



Non-Pipelined Processor

In the non-pipelined processor, one instruction must complete all five stages before the next instruction starts.

Instruction 1:

IF → ID → EX → MEM → WB

Instruction 2:

IF → ID → EX → MEM → WB

Instruction 3:

IF → ID → EX → MEM → WB

For 5 instructions:

$$T_{non-pipeline} = 5 \times 5$$

$$T_{non-pipeline} = 25 \text{ cycles}$$

Speedup

Speedup is:

$$Speedup = \frac{T_{non-pipeline}}{T_{pipeline}}$$

Therefore:

$$Speedup = \frac{25}{9}$$

$$Speedup = 2.78$$

So your pipeline is approximately 2.78 times faster for this example.

Throughput

Throughput means the number of completed instructions per cycle.

For the pipelined system:

$$Throughput = \frac{\text{Number of Instructions}}{\text{Total Cycles}}$$

$$= \frac{5}{9}$$

$$0.556 \text{ instructions/cycle}$$

For the non-pipelined system:

$$= \frac{5}{25}$$

$$0.2 \text{ instructions/cycle}$$

Thus, pipelining substantially improves throughput.

Result

The 5-stage pipeline was successfully simulated. Pipelining improved instruction throughput and reduced the average execution time compared with the non-pipelined system.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

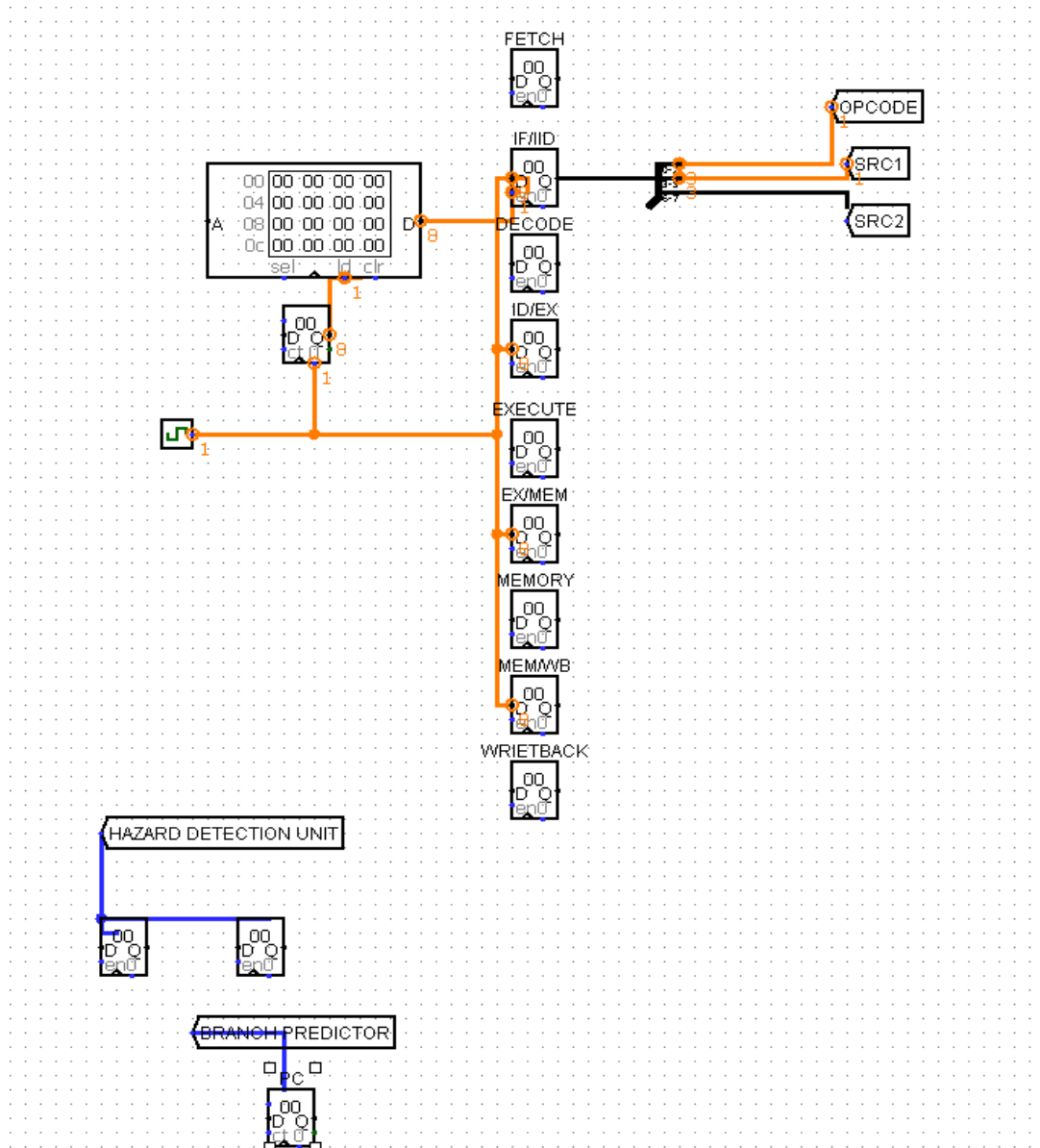
SOLUTION:

Aim:

To simulate data, control, and structural hazards in a pipeline and implement techniques to reduce their effects.

Algorithm:

- Define a sequence of dependent and branching instructions.
- Identify data hazards, control hazards, and structural hazards.
- Observe pipeline stalls caused by these hazards.
- Apply data forwarding to resolve data dependencies.
- Apply stalling when forwarding is insufficient.
- Use branch prediction to reduce control hazards.
- Compare execution before and after hazard handling.

Experimental design :

Expected Analysis**Data Hazard**

Without forwarding, dependent instructions require stalls. Forwarding reduces the waiting time by transferring results directly between pipeline stages.

Control Hazard

Branches can cause incorrect instructions to enter the pipeline. Branch prediction reduces this penalty by predicting the branch direction before the branch is fully resolved.

Structural Hazard

When two stages require the same hardware resource simultaneously, one instruction must wait. This can be handled by stalling or providing separate resources.

Result:

Pipeline hazards were successfully identified and resolved using stalling, data forwarding, and branch prediction, resulting in improved pipeline performance.

5. Analyse a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

SOLUTION:**Aim:**

To analyse superscalar execution with out-of-order and speculative execution and study the performance benefits of hyper-threading/SMT.

Algorithm:

- Define a set of independent and dependent instructions.
- Fetch and decode multiple instructions per cycle.
- Execute independent instructions in parallel using multiple execution units.
- Apply out-of-order execution to utilize available resources efficiently.
- Use speculative execution for predicted branches.
- Implement multiple threads using SMT/Hyper-Threading.
- Measure instruction throughput and resource utilization.
- Compare single-threaded and parallel execution.

Experimental Design:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation